

Лабораторная работа № 4

Эмуляция и измерение задержек в глобальных сетях

Шуваев Сергей Александрович

Содержание

1	Цель работы	4
2	Задание	5
3	Теоретическое введение	6
4	Выполнение лабораторной работы	7
4.1	Добавление/изменение задержки в эмулируемой глобальной сети	10
4.2	Изменение задержки в эмулируемой глобальной сети	12
4.3	Восстановление исходных значений (удаление правил) задержки в эмулируемой глобальной сети	13
4.4	Добавление значения дрожания задержки в интерфейс подключения к эмулируемой глобальной сети	14
4.5	Добавление значения корреляции для джиттера и задержки в интерфейс подключения к эмулируемой глобальной сети	15
4.6	Распределение задержки в интерфейсе подключения к эмулируемой глобальной сети	16
4.7	Воспроизведение экспериментов. Добавление задержки для интерфейса, подключающегося к эмулируемой глобальной сети	17
5	Выводы	22
	Список литературы	23

Список иллюстраций

4.1	Исправление прав запуска X-соединения	7
4.2	Простейшая топология	8
4.3	ifconfig на хостах h1 и h2	9
4.4	Проверка подключения между хостами	10
4.5	Добавление задержки в 100мс	11
4.6	Двунаправленная задержка соединения	12
4.7	Изменение задержки на 50мс	13
4.8	Восстановление исходных значений задержки	14
4.9	Добавление значения дрожания задержки в интерфейс подключения	14
4.10	Добавление значения корреляции для джиттера и задержки в ин- терфейс подключения	15
4.11	Распределение задержки в интерфейсе подключения	16
4.12	скрипт для визуализации ping_plot	19
4.13	Makefile для управления процессом проведения эксперимента . .	19
4.14	результате выполнения скриптов	20
4.15	результате выполнения скрипта	21
4.16	Скрипт rtt.py	21
4.17	Результат работы скрипта rtt.py	21

1 Цель работы

Основной целью работы является знакомство с NETEM — инструментом для тестирования производительности приложений в виртуальной сети, а также получение навыков проведения интерактивного и воспроизводимого экспериментов по измерению задержки и её дрожания (jitter) в моделируемой сети в среде Mininet.

2 Задание

1. Задайте простейшую топологию, состоящую из двух хостов и коммутатора с назначенной по умолчанию mininet сетью 10.0.0.0/8.
2. Проведите интерактивные эксперименты по добавлению/изменению задержки, джиттера, значения корреляции для джиттера и задержки, распределения времени задержки в эмулируемой глобальной сети.
3. Реализуйте воспроизводимый эксперимент по заданию значения задержки в эмулируемой глобальной сети. Постройте график.
4. Самостоятельно реализуйте воспроизводимые эксперименты по изменению задержки, джиттера, значения корреляции для джиттера и задержки, распределения времени задержки в эмулируемой глобальной сети. Постройте графики.

3 Теоретическое введение

Mininet[**mininet?**] – это эмулятор компьютерной сети. Под компьютерной сетью подразумеваются простые компьютеры — хосты, коммутаторы, а так же OpenFlow-контроллеры. С помощью простейшего синтаксиса в примитивном интерпретаторе команд можно разворачивать сети из произвольного количества хостов, коммутаторов в различных топологиях и все это в рамках одной виртуальной машины(ВМ). На всех хостах можно изменять сетевую конфигурацию, пользоваться стандартными утилитами(`ifconfig`, `ping`) и даже получать доступ к терминалу. На коммутаторы можно добавлять различные правила и маршрутизировать трафик.

4 Выполнение лабораторной работы

Запустим виртуальную среду с mininet. Из основной ОС подключимся к виртуальной машине. В виртуальной машине mininet при необходимости исправим права запуска X-соединения. Скопируем значение куки (MIT magic cookie) своего пользователя mininet в файл для пользователя root (рис. 4.1).

```
Last login: Wed Oct 22 12:49:07 2025 from 192.168.56.1
mininet@mininet-vm:~$ xauth list $DISPLAY
mininet-vm/unix:11 MIT-MAGIC-COOKIE-1 7de23c785508816a968628d6787dcdd1
mininet@mininet-vm:~$ sudo -i
root@mininet-vm:~# xauth add mininet-vm/unix:10 MIT-MAGIC-COOKIE-1 295acad8e35d1
7636924c5ab80e8462d
root@mininet-vm:~# logout
mininet@mininet-vm:~$
```

Рис. 4.1: Исправление прав запуска X-соединения

Задайте простейшую топологию, состоящую из двух хостов и коммутатора с назначенной по умолчанию mininet сетью 10.0.0.0/8 (рис. 4.2).

После введения этой команды запустятся терминалы двух хостов, коммутатора и контроллера. Терминалы коммутатора и контроллера можно закрыть.

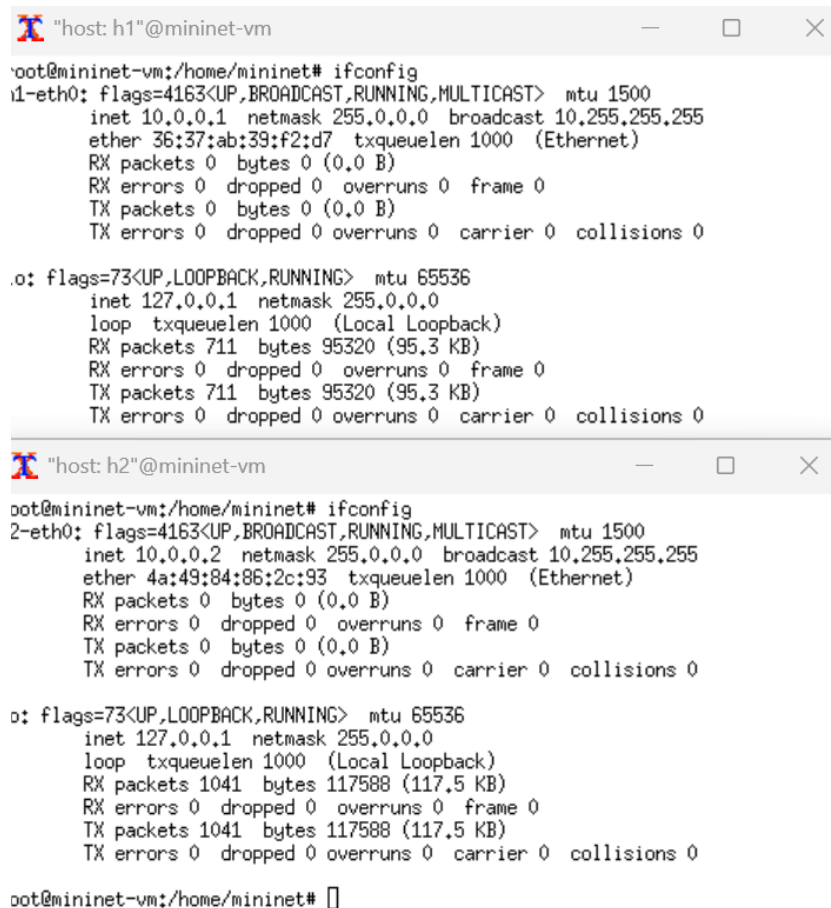
```

mininet@mininet-vm:~$ sudo mn --topo=single,2 -x
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Running terms on localhost:11.0
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> 

```

Рис. 4.2: Простейшая топология

На хостах h1 и h2 введем команду `ifconfig`, чтобы отобразить информацию, относящуюся к их сетевым интерфейсам и назначенным им IP-адресам. В дальнейшем при работе с NETEM и командой `tc` будут использоваться интерфейсы h1-eth0 и h2-eth0 (рис. 4.3).



The image shows two terminal windows from a Mininet VM. The top window is titled '"host: h1"@mininet-vm' and shows the output of the 'ifconfig' command. It displays details for the 'v1-eth0' interface (Ethernet) and the 'lo' interface (Local Loopback). The bottom window is titled '"host: h2"@mininet-vm' and shows the output of the 'ifconfig' command for host h2, displaying details for the '2-eth0' interface and the 'lo' interface. Both windows show that the interfaces are up and running, with no errors or collisions reported.

```
"host: h1"@mininet-vm
root@mininet-vm:/home/mininet# ifconfig
v1-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.0.1 netmask 255.0.0.0 broadcast 10.255.255.255
    ether 36:37:ab:39:f2:d7 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 711 bytes 95320 (95.3 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 711 bytes 95320 (95.3 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

"host: h2"@mininet-vm
root@mininet-vm:/home/mininet# ifconfig
2-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.0.2 netmask 255.0.0.0 broadcast 10.255.255.255
    ether 4a:49:84:86:2c:93 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 1041 bytes 117588 (117.5 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1041 bytes 117588 (117.5 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@mininet-vm:/home/mininet#
```

Рис. 4.3: ifconfig на хостах h1 и h2

Проверим подключение между хостами h1 и h2 с помощью команды ping с параметром -c 6 (рис. 4.4).

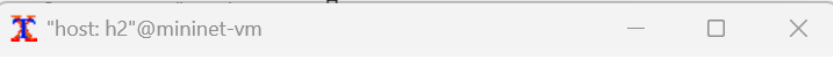
```

RX packets 711 bytes 95520 (95.5 KB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@mininet-vm:/home/mininet# ping -c 6 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=1.68 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.276 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.046 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.048 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.040 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.040 ms

--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5102ms
rtt min/avg/max/mdev = 0.040/0.355/1.681/0.598 ms

```



```

TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 1041 bytes 117588 (117.5 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1041 bytes 117588 (117.5 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@mininet-vm:/home/mininet# ping -c 6 10.0.0.1^C
root@mininet-vm:/home/mininet# ping -c 6 10.0.0.1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data:
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=0.828 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=0.038 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=0.037 ms
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=0.036 ms
64 bytes from 10.0.0.1: icmp_seq=5 ttl=64 time=0.038 ms
64 bytes from 10.0.0.1: icmp_seq=6 ttl=64 time=0.045 ms

--- 10.0.0.1 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5120ms
rtt min/avg/max/mdev = 0.036/0.170/0.828/0.294 ms
root@mininet-vm:/home/mininet#

```

Рис. 4.4: Проверка подключения между хостами

4.1 Добавление/изменение задержки в эмулируемой глобальной сети

На хосте h1 добавим задержку в 100 мс к выходному интерфейсу (рис. 4.5).

```
sudo tc qdisc add dev h1-eth0 root netem delay 100ms
```

- `sudo`: выполнить команду с более высокими привилегиями;
- `tc`: вызвать управление трафиком Linux;
- `qdisc`: изменить дисциплину очередей сетевого планировщика;
- `add`: создать новое правило;
- `dev h1-eth0`: указать интерфейс, на котором будет применяться правило;

- netem: использовать эмулятор сети;
- delay 100ms: задержка ввода 100 мс.

Проверим, что соединение от хоста h1 к хосту h2 имеет задержку 100 мс, используя команду ping с параметром -c 6 с хоста h1

```
root@mininet-vm:/home/mininet# sudo tc qdisc add dev h1-eth0 root r
00ms
root@mininet-vm:/home/mininet# ping -c 6 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=100 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=105 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=100 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=102 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=107 ms

--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 501ms
rtt min/avg/max/mdev = 100.208/102.466/106.869/2.537 ms
root@mininet-vm:/home/mininet#
```

Рис. 4.5: Добавление задержки в 100мс

Для эмуляции глобальной сети с двунаправленной задержкой необходимо к соответствующему интерфейсу на хосте h2 также добавим задержку в 100 миллисекунд (рис. 4.6).

Проверим, что соединение между хостом h1 и хостом h2 имеет RTT в 200 мс (100 мс от хоста h1 к хосту h2 и 100 мс от хоста h2 к хосту h1), повторив команду ping с параметром -c 6 на терминале хоста h1.

```
"host: h2"@mininet-vm
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 1041 bytes 117588 (117.5 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1041 bytes 117588 (117.5 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@mininet-vm:/home/mininet# ping -c 6 10.0.0.1^C
root@mininet-vm:/home/mininet# ping -c 6 10.0.0.1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data:
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=0.828 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=0.038 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=0.037 ms
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=0.036 ms
64 bytes from 10.0.0.1: icmp_seq=5 ttl=64 time=0.038 ms
64 bytes from 10.0.0.1: icmp_seq=6 ttl=64 time=0.045 ms

--- 10.0.0.1 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5120ms
rtt min/avg/max/mdev = 0.036/0.170/0.828/0.294 ms
root@mininet-vm:/home/mininet# sudo tc qdisc add dev h2-eth0 root netem delay 1
00ms
root@mininet-vm:/home/mininet# █

--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5126ms
rtt min/avg/max/mdev = 200.715/233.230/372.780/62.500 ms
root@mininet-vm:/home/mininet# █
```

Рис. 4.6: Двухнаправленная задержка соединения

4.2 Изменение задержки в эмулируемой глобальной сети

Изменим задержку со 100 мс до 50 мс для отправителя h1 и для получателя h2 (рис. 4.7).

Проверим, что соединение от хоста h1 к хосту h2 имеет задержку 100 мс, используя команду ping с параметром -c 6 с терминала хоста h1.

```
Cannot find device "h1-eth0"
root@mininet-virtual-machine:/home/mininet# ^C
root@mininet-virtual-machine:/home/mininet#
root@mininet-virtual-machine:/home/mininet# sudo tc qdisc change dev h2-eth0 root netem delay 50ms
root@mininet-virtual-machine:/home/mininet# ping -c 6 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
 64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=186 ms
 64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=197 ms
 64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=166 ms
 64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=195 ms
 64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=157 ms
 64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=152 ms

--- 10.0.0.2 ping statistics ---
 6 packets transmitted, 6 received, 0% packet loss, time 5018ms
rtt min/avg/max/mdev = 152.100/175.558/197.325/17.989 ms
root@mininet-virtual-machine:/home/mininet#
```

Рис. 4.7: Изменение задержки на 50мс

4.3 Восстановление исходных значений (удаление правил) задержки в эмулируемой глобальной сети

Восстановим конфигурацию по умолчанию, удалив все правила, применённые к сетевому планировщику соответствующего интерфейса для отправителя h1 и для получателя h2. Проверим, что соединение между хостом h1 и хостом h2 не имеет явно установленной задержки, используя команду `ping` с параметром `-c 6` с терминала хоста h1 (рис. 4.8).

```
root@mininet-vm:/home/mininet# sudo tc qdisc del dev h2-eth0 root netem
root@mininet-vm:/home/mininet# 
"host: h1"@mininet-vm
Unknown qdisc "h1-eth0", hence option "root" is unparsable
root@mininet-vm:/home/mininet# sudo tc qdisc del h2-eth0 root netem
Unknown qdisc "h2-eth0", hence option "root" is unparsable
root@mininet-vm:/home/mininet# sudo tc qdisc del dev h1-eth0 root netem
root@mininet-vm:/home/mininet# ping -c 6 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.903 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.524 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.192 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.046 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.038 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.038 ms

--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5131ms
rtt min/avg/max/mdev = 0.038/0.290/0.903/0.323 ms
root@mininet-vm:/home/mininet#
```

Рис. 4.8: Восстановление исходных значений задержки

4.4 Добавление значения дрожания задержки в интерфейс подключения к эмулируемой глобальной сети

Добавим на узле h1 задержку в 100 мс со случайным отклонением 10 мс. Проверим, что соединение от хоста h1 к хосту h2 имеет задержку 100 мс со случайным отклонением ± 10 мс, используя в терминале хоста h1 команду ping с параметром -c 6. Восстановим конфигурацию интерфейса по умолчанию на узле h1 (рис. 4.9).

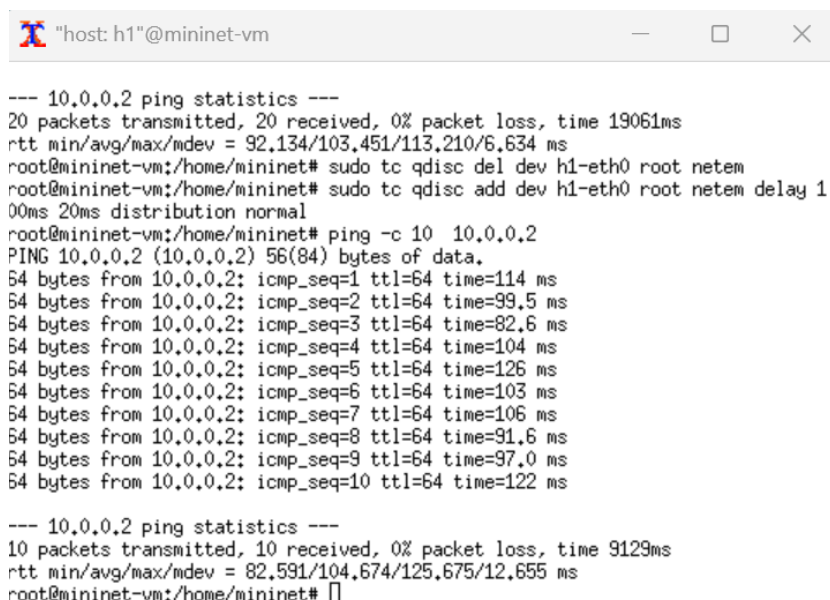
```
"host: h1"@mininet-vm
5 packets transmitted, 6 received, 0% packet loss, time 5131ms
rtt min/avg/max/mdev = 0.038/0.290/0.903/0.323 ms
root@mininet-vm:/home/mininet# sudo tc qdisc add dev h1-eth0 root netem delay 100ms 10ms
root@mininet-vm:/home/mininet# ping -c 6 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
34 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=102 ms
34 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=99.3 ms
34 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=102 ms
34 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=94.3 ms
34 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=106 ms
34 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=104 ms

--- 10.0.0.2 ping statistics ---
5 packets transmitted, 6 received, 0% packet loss, time 5006ms
rtt min/avg/max/mdev = 94.342/101.296/105.678/3.671 ms
root@mininet-vm:/home/mininet#
```

Рис. 4.9: Добавление значения дрожания задержки в интерфейс подключения

4.5 Добавление значения корреляции для джиттера и задержки в интерфейс подключения к эмулируемой глобальной сети

Добавим на интерфейсе хоста h1 задержку в 100 мс с вариацией ± 10 мс и значением корреляции в 25%. Убедимся, что все пакеты, покидающие устройство h1 на интерфейсе h1-eth0, будут иметь время задержки 100 мс со случайным отклонением ± 10 мс, при этом время передачи следующего пакета зависит от предыдущего значения на 25%. Используем для этого в терминале хоста h1 команду ping с параметром -c 20. Восстановим конфигурацию интерфейса по умолчанию на узле h1 (рис. 4.10).



```
--- 10.0.0.2 ping statistics ---
20 packets transmitted, 20 received, 0% packet loss, time 19061ms
rtt min/avg/max/mdev = 92.134/103.451/113.210/6.634 ms
root@mininet-vm:/home/mininet# sudo tc qdisc del dev h1-eth0 root netem
root@mininet-vm:/home/mininet# sudo tc qdisc add dev h1-eth0 root netem delay 1
00ms 20ms distribution normal
root@mininet-vm:/home/mininet# ping -c 10 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data:
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=114 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=99,5 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=82,6 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=104 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=126 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=103 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=106 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=91,6 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=97,0 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=122 ms

--- 10.0.0.2 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9129ms
rtt min/avg/max/mdev = 82,591/104,674/125,675/12,655 ms
root@mininet-vm:/home/mininet#
```

Рис. 4.10: Добавление значения корреляции для джиттера и задержки в интерфейс подключения

4.6 Распределение задержки в интерфейсе подключения к эмулируемой глобальной сети

Зададим нормальное распределение задержки на узле h1 в эмулируемой сети. Убедимся, что все пакеты, покидающие хост h1 на интерфейсе h1-eth0, будут иметь время задержки, которое распределено в диапазоне 100 мс $\pm$ 20 мс. Используем для этого команду ping на терминале хоста h1 с параметром -с 10. Восстановим конфигурацию интерфейса по умолчанию на узле h1. Завершим работу mininet в интерактивном режиме (рис. 4.11).

```
#!/usr/bin/env python

"""
Simple experiment.
Output: ping.dat
"""

from mininet.net import Mininet
from mininet.node import Controller
from mininet.cli import CLI
from mininet.log import setLogLevel, info
import time

def emptyNet():
    "Create an empty network and add nodes to it."
    net = Mininet(controller=Controller,
                  waitConnected=True)

    info( "**** Adding controller\n" )
    net.addController( 'c0' )

    info( "**** Adding hosts\n" )
    h1 = net.addHost( 'h1', ip='10.0.0.1' )
    h2 = net.addHost( 'h2', ip='10.0.0.2' )

    info( "**** Adding switch\n" )
    s1 = net.addSwitch( 's1' )

    info( "**** Creating links\n" )
    net.addLink( h1, s1 )
    net.addLink( h2, s1 )

    info( "**** Starting network\n" )
    net.start()

    info( "**** Set delay\n" )
    h1.cmdPrint( 'tc qdisc add dev h1-eth0 root netem delay 100ms' )
    h2.cmdPrint( 'tc qdisc add dev h2-eth0 root netem delay 100ms' )

    time.sleep(10) # Wait 10 seconds

    info( "**** Ping\n" )
    h1.cmdPrint( 'ping -c 100', h2.IP(), '| grep "time=" | awk \'{print $5, $7}\'} | sed -e \'/s/time=//g\' -e \'/s/icmp_seq=//g\' > ping.dat' )

    info( "**** Stopping network" )
    net.stop()

if __name__ == '__main__':
    setLogLevel( 'info' )
    emptyNet()
```

Рис. 4.11: Распределение задержки в интерфейсе подключения

4.7 Воспроизведение экспериментов. Добавление задержки для интерфейса, подключающегося к эмулируемой глобальной сети

С помощью API Mininet воспроизведем эксперимент по добавлению задержки для интерфейса хоста, подключающегося к эмулируемой глобальной сети. В виртуальной среде mininet в своём рабочем каталоге с проектами создадим каталог simple-delay и перейдем в него.

Создадим скрипт для эксперимента lab_netem_i.py:

```
1 #!/usr/bin/env python
2
3 """
4 Simple experiment.
5 Output: ping.dat
6 """
7
8 from mininet.net import Mininet
9 from mininet.node import Controller
10 from mininet.cli import CLI
11 from mininet.log import setLogLevel, info
12 import time
13
14 def emptyNet():
15
16     "Create an empty network and add nodes to it."
17
18     net = Mininet( controller=Controller,
19 ↪ waitConnected=True )
```

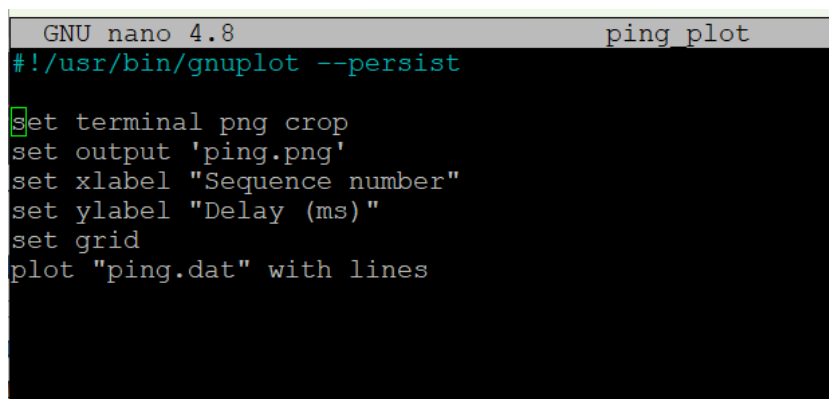
```

19
20 info( '*** Adding controller\n' )
21 net.addController( 'c0' )
22
23 info( '*** Adding hosts\n' )
24 h1 = net.addHost( 'h1', ip='10.0.0.1' )
25 h2 = net.addHost( 'h2', ip='10.0.0.2' )
26
27 info( '*** Adding switch\n' )
28 s1 = net.addSwitch( 's1' )
29
30 info( '*** Creating links\n' )
31 net.addLink( h1, s1 )
32 net.addLink( h2, s1 )
33
34 info( '*** Starting network\n' )
35 net.start()
36
37 info( '*** Set delay\n' )
38 h1.cmdPrint( 'tc qdisc add dev h1-eth0 root netem delay 100ms' )
39 h2.cmdPrint( 'tc qdisc add dev h2-eth0 root netem delay 100ms' )
40
41 time.sleep(10) # Wait 10 seconds
42
43 info( '*** Ping\n' )
44 h1.cmdPrint( 'ping -c 100', h2.IP(), '| grep "time=" | awk \'{print $5, $7}\'' | sed
45
46 info( '*** Stopping network' )
47 net.stop()

```

```
48
49 if __name__ == '__main__':
50     setLogLevel( 'info' )
51     emptyNet()
```

Создаём скрипт для визуализации ping_plot результатов эксперимента (рис. 4.12).



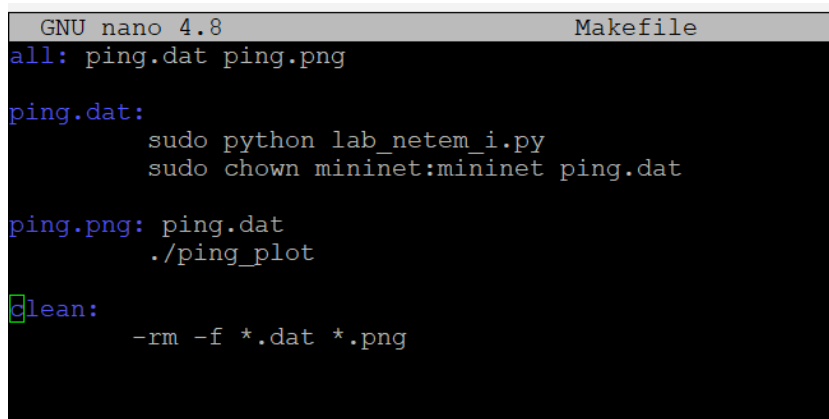
```
GNU nano 4.8 ping_plot
#!/usr/bin/gnuplot --persist

set terminal png crop
set output 'ping.png'
set xlabel "Sequence number"
set ylabel "Delay (ms)"
set grid
plot "ping.dat" with lines
```

Рис. 4.12: скрипт для визуализации ping_plot

Зададим права доступа к файлу скрипта: `chmod +x ping_plot`.

Создадим Makefile для управления процессом проведения эксперимента (рис. 4.13).



```
GNU nano 4.8 Makefile
all: ping.dat ping.png

ping.dat:
    sudo python lab_netem_i.py
    sudo chown mininet:mininet ping.dat

ping.png: ping.dat
    ./ping_plot

clean:
    -rm -f *.dat *.png
```

Рис. 4.13: Makefile для управления процессом проведения эксперимента

Выполним эксперимент. Продемонстрируем построенный в результате выполнения скриптов график (рис. 4.14).

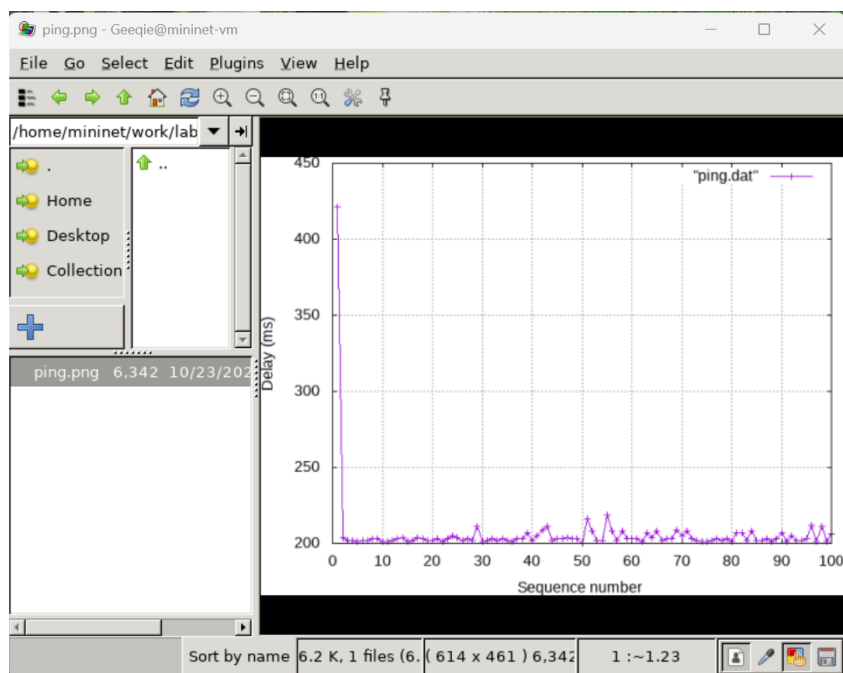


Рис. 4.14: результате выполнения скриптов

Из файла ping.dat удалим первую строку и заново постройте график. Продемонстрируем построенный в результате график (рис. 4.15).

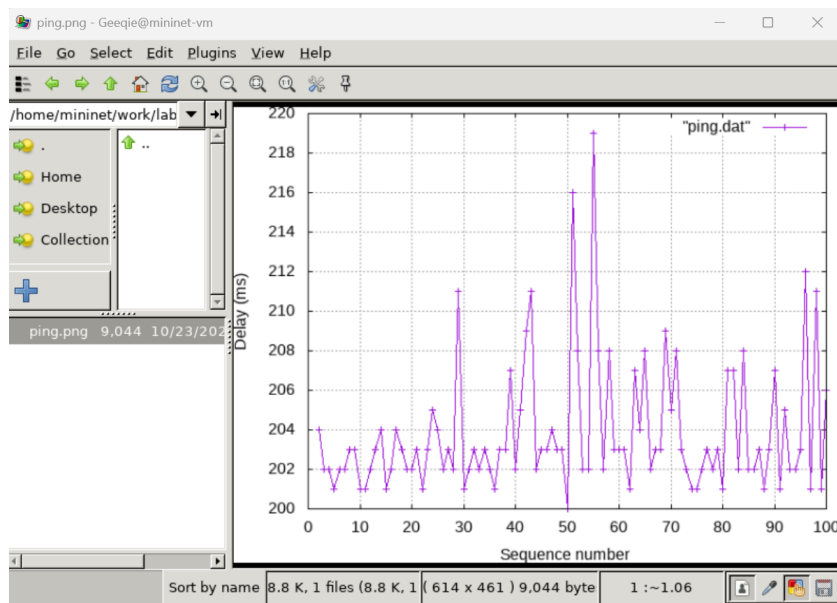


Рис. 4.15: результате выполнения скрипта

Разработайте скрипт для вычисления на основе данных файла ping.dat минимального, среднего, максимального и стандартного отклонения времени приёма-передачи (рис. 4.16).

```
GNU nano 4.8                                rtt.py
with open('ping.dat', 'r') as f:
    s = []
    for line in f.readlines():
        if '\n' in line:
            line.replace('\n', "")
            s.append([int(j) for j in (line.split(" "))])
    s = [j[1] for j in s]
    std = (sum([(i - (sum(s)/len(s)))**2 for i in s])/(len(s)-1))**0.5
    print(f"min: {min(s)} \max: {max(s)} \n avg: {sum(s)/len(s)} \n std: {std}")
```

Рис. 4.16: Скрипт rtt.py

Продемонстрируем работу скрипта с выводом значений на экран или в отдельный файл (рис. 4.17).

```
mininet@mininet-vm:~/work/lab_netem_i/simple-delay$ nano rtt.py
mininet@mininet-vm:~/work/lab_netem_i/simple-delay$ sudo python rtt.py
min: 200 \max: 219
avg: 203.8181818181818
std: 3.2899464556266746
```

Рис. 4.17: Результат работы скрипта rtt.py

5 Выводы

В результате выполнения данной лабораторной работы я познакомился с NETEM – инструментом для тестирования производительности приложений в виртуальной сети, а также получил навыки проведения интерактивного и воспроизводимого экспериментов по измерению задержки и её дрожания (jitter) в моделируемой сети в среде Mininet.

Список литературы